

Prompts

.1 Stage I: Requirements Extraction Prompt

Listing 1: First-iteration requirement extraction prompt for Stage 1.

```
# Objective
  You are an expert software requirements analyst. Extract functional requirements,
  quality requirements, constraints and acceptance criteria from the provided
  software document.

# Instructions
  1. Review the entire document.
  2. Identify all functional requirements (FR), quality requirements (QR), and
  constraints and their acceptance criteria for all of them.
  3. For each identified FR, QR, constraint and criterion apply the specific
  processing logic found in the Extraction Process by Type section below, during the
  extraction make sure to oblige the rules defined in Rules section.

# Extraction Process by Type

## For All Types
  - Assign a unique, strict sequential ID starting from R_01 (e.g., R_01, R_02,
  R_03...) regardless of type.
  - Specify the page number from the document text (not the PDF page number) where
  the requirement starts.
  - Specify the type (FR, QR, constraint or criterion).

## For FR, QR and Constraint
  - In the case of document stating incorrect type correct it and specify the fix

## For FR
  - In the case of a FR expressed as a user story AS A actor I WANT something IN
  ORDER TO whatever, extract I WANT something.
  - In the case of a FR expressed in a use case specification, extract the objective.
  - In the case of a FR expressed as a plain text without a specific format, FR is
  the text.
  - Extracted FR should have the following JSON Schema:
    {
      "id": "<Sequential Id shared with all types (R_01, R_02)>",
      "type": "<FR>",
      "description": "<FR in English>",
      "pageNumber": "<Page number of the FR>",
      "fix": "<Changed from ... to FR>"
    }

## For QR
  - For QRs, concept and categorization should be extracted from the document.
```

- In case of a QR expressed using Volere, the concept is the QR type written in the Requirement part and categorization is Volere.
- In case of a QR classified from ISO/IEC 25010, the concept is the subcharacteristic or, in case it is not stated, the characteristic. And the categorization is ISO/IEC 25010.
- In case of a QR expressed using Volere with Description and Acceptance, extract the Description contents.
- In the case of a QR expressed as a plain text without a specific format, QR is the text.

- Extracted QR should have the following JSON schema:

```
{
  "id": "<Sequential Id shared with all types (R_01, R_02)>",
  "type": "<QR>",
  "description": "<QR in English>",
  "pageNumber": "<Page number of the requirement>",
  "concept": "<Type of QR (in English) >",
  "categorization": "<Volere or ISO/IEC 25010>",
  "fix": "<Changed from ... to QR>"
}
```

For Constraint

- Extracted constraint should have the following JSON Schema:

```
{
  "id": "<Sequential Id shared with all types (R_01, R_02)>",
  "type": "<Constraint>",
  "description": "<Constraint in English>",
  "pageNumber": "<Page number of the Constraint>",
  "fix": "<Changed from ... to Constraint>"
}
```

For Criterion

- Acceptance criteria should be extracted from acceptance criteria section for each requirement (if exist).
- In case of a QR expressed using Volere with several acceptance criterion, each should be added as a separate criterion.
- In the case of FR expressed as a user story with several acceptance criterion, each should be added as separate criterion.
- The link between an acceptance criterion and its related requirement should be explicitly represented by stating the ID of the related requirement.
- Extracted criterion should have the following JSON schema:

```
{
  "id": "<Sequential Id shared with all types (R_01, R_02)>",
  "type": "<criterion>",
  "description": "<Actual acceptance criterion of the requirement in English>",
  "pageNumber": "<Page number of the criterion>",
  "relatedTo": "<Id of the related requirement>"
}
```

Rules:

- Ensure that all information is strictly supported by the document.
- Output should be given in JSON format as in Example Output section.
- Whole output must be English. If the source document is in another language, translation should be made while extracting to ensure all extracted fields are in English.

Example Output (JSON)

```
{
```

```

    "id": "R_01",
    "type": "FR",
    "description": "The system shall allow users to reset their password via email
    verification.",
    "pageNumber": "12",
    "fix": "Changed from QR to FR"
  },
  {
    "id": "R_02",
    "type": "criterion",
    "description": "When a registered user requests a password reset, the system
    shall send a password reset email containing a valid verification link within 1
    minute.",
    "pageNumber": "12",
    "relatedTo": "R_01",
  },
  {
    "id": "R_03",
    "type": "QR",
    "description": "The system shall respond to user requests within 2 seconds under
    normal operating conditions.",
    "pageNumber": "15",
    "concept": "12a. Speed and latency",
    "categorization": "Volere"
  },
  {
    "id": "R_04",
    "type": "criterion",
    "description": "During performance testing with up to 1,000 concurrent users, 95%
    of requests shall complete within 2 seconds.",
    "pageNumber": "15",
    "relatedTo": "R_03"
  }
  ...

```

Listing 2: Final requirement extraction prompt for Stage I.

```

# Objective
You are an expert software requirements analyst. Extract functional requirements,
quality requirements, constraints and acceptance criteria from the provided
software document.

# Instructions
1. Review the entire document.
2. Identify all functional requirements (FR), quality requirements (QR), and
constraints and their acceptance criteria for all of them.
3. For each identified FR, QR, constraint and criterion apply the specific
processing logic found in the Extraction Process by Type section below, during the
extraction make sure to oblige the rules defined in Rules section.

# Extraction Process by Type

## For All Types
- Assign a unique, strict sequential ID starting from R_01 (e.g., R_01, R_02,
R_03...) regardless of type.
- Specify the page number from the document text (not the PDF page number) where
the requirement starts.
- Specify the type (FR, QR, constraint or criterion).

```

```

## For FR, QR and Constraint
- In the case of document stating incorrect type correct it and specify the changed
  type in fixedType field.

## For FR
- In the case of a FR expressed as a user story AS A actor I WANT something IN
  ORDER TO whatever, extract I WANT something.
- In the case of a FR expressed in a use case specification, extract the objective.
- In the case of a FR expressed as a plain text without a specific format, FR is
  the text.
- Extracted FR should have the following JSON Schema:
  {
    "id":<Sequential Id shared with all types (R_01, R_02)>,
    "type": "<FR>",
    "description": "<FR in English>",
    "pageNumber": "<Page number of the FR>",
    "fixedType": "<If fixed QR or constraint, else empty>"
  }

## For QR
- For QRs, concept and categorization should be extracted from the document.
- In case of a QR expressed using Volere with Description and Acceptance, extract
  the Description contents.
- In the case of a QR expressed as a plain text without a specific format, QR is
  the text.
- Extracted QR should have the following JSON schema:
  {
    "id":<Sequential Id shared with all types (R_01, R_02)>,
    "type": "<QR>",
    "description": "<QR in English>",
    "pageNumber": "<Page number of the requirement>",
    "concept": "<Type of QR (in English) >",
    "fixedType": "<If fixed, FR or constraint else empty>"
  }

## For Constraint
- Extracted constraint should have the following JSON Schema:
  {
    "id":<Sequential Id shared with all types (R_01, R_02)>,
    "type": "<Constraint>",
    "description": "<Constraint in English>",
    "pageNumber": "<Page number of the Constraint>",
    "fixedType": "<If fixed, QR or FR else empty>"
  }

## For Criterion
- Acceptance criteria should be extracted from acceptance criteria section for each
  requirement (if exists).
- In case of a QR expressed using Volere with several acceptance criterion, each
  should be added as a separate criterion.
- In the case of FR expressed as a user story with several acceptance criterion,
  each should be added as separate criterion.
- The link between an acceptance criterion and its related requirement should be
  explicitly represented by stating the ID of the related requirement.
- Extracted criterion should have the following JSON schema:
  {
    "id":<Sequential Id shared with all types (R_01, R_02)>,

```

```

        "type": "<criteria>",
        "description": "<Actual acceptance criterion of the requirement in English
        >>",
        "pageNumber": "<Page number of the criterion>",
        "relatedTo": "<Id of the related requirement>"
    }

# Rules:
- Ensure that all information is strictly supported by the document.
- Output should be given in JSON format as in Example Output section.
- Whole output must be English. If the source document is in another language,
translation should be made while extracting to ensure all extracted fields are in
English.

# Example Output (JSON)
{
  "id": "R_01",
  "type": "FR",
  "description": "The system shall allow users to reset their password via email
  verification.",
  "pageNumber": "12",
  "fixedType": "QR"
},
{
  "id": "R_02",
  "type": "criterion",
  "description": "When a registered user requests a password reset, the system
  shall send a password reset email containing a valid verification link within 1
  minute.",
  "pageNumber": "12",
  "relatedTo": "R_01",
},
{
  "id": "R_03",
  "type": "QR",
  "description": "The system shall respond to user requests within 2 seconds under
  normal operating conditions.",
  "pageNumber": "15",
  "concept": "12a. Speed and latency",
  "fixedType": ""
},
{
  "id": "R_04",
  "type": "criterion",
  "description": "During performance testing with up to 1,000 concurrent users, 95%
  of requests shall complete within 2 seconds.",
  "pageNumber": "15",
  "relatedTo": "R_03"
}
...

```

.2 Stage II: Architectural Element Extraction Prompt

Listing 3: Architectural Element Extraction Prompt v1

```
# Objective
```

You are an expert software architect and system design analyst. Extract the Architectural Units (the concrete building blocks of the system) and patterns from the provided software document.

Instructions

1. Carefully read the entire document.
2. Identify architectural units and patterns using the type definitions below.
3. Extract the identified architectural units and patterns following the extraction process defined below.

Type Definitions

Below are the definitions for each type of architectural unit and pattern with their concrete examples.

Patterns

Architectural Pattern

- Definition: A high-level structural organization of the system (e.g. Client-Server, Layered Architecture, Microservices, MVVM, Service-oriented, Cloud Architecture).
- Examples: Layered Architecture, Hexagonal Architecture, API Gateway.

Design Pattern

- Definition: A lower-level software design solution used within units (e.g. API Gateway, Repository, Observer, Singleton, Shared Database, ORM, Component-based).
- Examples: Observer, Strategy, Factory.

Architectural Units

Layer

- Definition: a horizontal tier that groups units by a shared responsibility in a layered / n-tier architecture.
- Examples: presentation layer, business layer, data layer.

Component

- Definition: a concrete structural module of the system that either:
 1. represents an internal functional or structural part of the system that is not exposed as an independently running service; OR
 2. represents a technology-independent architectural role fulfilled by an infrastructure or data-management element.
- Examples: View, ViewModel, Model, Controller, Repository, Cache, Message Broker, Database.

Service

- Definition: a logical capability or independently running module of the system, including each microservice and each external / third-party service the system depends on, integrates with, or calls.
- Examples: API gateway, Microservice, Authentication Service, Payment Service.

Device

- Definition: a physical hardware endpoint or piece of equipment that participates in the system.
- Examples: a sensor, a screen / display, a kiosk or player device, a mobile or desktop device, an IoT device, a hardware appliance.

Technology

- Definition: a specific, named product, framework, library, programming language, protocol, cloud service, or development / testing / monitoring tool used to build

or run the system, including a named third-party product that provides an external service or integration.

- Examples: PostgreSQL, React, AWS S3, Python, Spring Boot.

Connector

- Definition: a communication or interaction relationship among two or more Architectural Units. It can link any unit type to any unit type (e.g. layer-layer, service-service, service-database, device-service, component-service ...). A Connector has no name.

- One-to-one: a communication stated between two units.

- One-to-many: a single stated communication in which one unit communicates with several other units (e.g. a unit that routes, distributes, logs, or mediates communication among many units).

Other

- Definition: a participant in the system's architecture that does not fit any technical type above; might be:

1. an external actor that interacts with the system; OR

2. a high-level logical or structural part of the system that groups lower-level architectural elements but is not itself treated as a Layer, Component or Service.

- Examples: an end user, a client, "the backend", "the frontend".

Extraction Process

Patterns

- Assign each Pattern a strict sequential id: P_01, P_02, P_03...

- For each pattern, specify the page number from the document text (not the PDF page number) where it is described. If it spans several pages, list them all.

- For each pattern "description" must be the sentence or sentences from the document that states the pattern - the evidence proving the pattern.

- For each pattern, specify its type using exactly one of:

- "isPartOf" is the list of OTHER pattern or unit ids this pattern belongs to.

Leave it as an empty list when no parent pattern applies.

- If the source document misclassifies a pattern (e.g. a Design Pattern labelled as an Architectural Pattern), correct the classification and specify the changed type

- Pattern should have the following JSON Schema:

```
{
  "id": "<Sequential Pattern id (P_01, P_02)>",
  "type": "<Architectural Pattern | Design Pattern>",
  "name": "<Name of the pattern in English>",
  "description": "<The exact document sentence(s) stating the pattern, translated to English>",
  "pageNumber": "<Page number(s) where the pattern is described>",
  "isPartOf": ["<id of the pattern this pattern is part of>"],
  "fixedType": "<If fixed, indicate the type before the fix>"
}
```

Architectural Units

- Assign each Architectural Unit a strict sequential id: AU_01, AU_02, AU_03...

- Specify its type using exactly one of: "Layer", "Component", "Service", "Device", "Technology", "Connector" or "Other".

- Specify description, sentence or sentences from the document that states the unit
- the evidence proving the unit.

- Specify the page number from the document text (not the PDF page number) where it is described. If it spans several pages, list them all.
- For each unit except connector, "isPartOf" is the architectural unit or a pattern (architectural or design) this unit is contained by or belongs to. If there is no unit or pattern applies, leave it empty.
- For connector, "isPartOf" is the list of the TWO OR MORE Architectural Unit ids (AU_xx, taken from the provided units) that the Connector links. Include exactly the units the stated communication involves. For a one-to-many communication, list the central (hub) unit FIRST, followed by every unit it communicates with.
- Architectural Unit should have the following JSON Schema:

```
{
  "id": "<Sequential Architectural Unit id (AU_01, AU_02)>",
  "type": "<Layer | Component | Service | Device | Technology | Connector | Other>",
  "name": "<Name of the unit (If connector then empty) >",
  "description": "<The exact document sentence(s) stating the unit, translated to English>",
  "pageNumber": "<Page number(s) where the unit is described>",
  "isPartOf": ["<id of the unit this unit is part of>"],
}
```

Rules:

- Output should be given in JSON format as in the Example Output section.
- The whole output must be English. If the source document is in another language, translation should be made while extracting to ensure all extracted fields are in English.
 - During translating, avoid to paraphrase, summarize, reword, or normalize phrasing.
- Avoid adding any extra explanation, just provide the required data.

Example Output (JSON)

```
{
  "architectural_units": [
    {
      "id": "AU_01",
      "type": "Other",
      "name": "Client",
      "description": "The client is the user of the platform.",
      "pageNumber": "41",
      "isPartOf": [],
    },
    {
      "id": "AU_02",
      "type": "Other",
      "name": "Backend",
      "description": "The Backend is the software that is responsible for processing user requests.",
      "pageNumber": "41",
      "isPartOf": [P_01],
    },
    {
      "id": "AU_03",
      "type": "Layer",
      "name": "Presentation Layer",
      "description": "It is the layer that is responsible for displaying information to the user.",
      "pageNumber": "41",
      "isPartOf": [],
    }
  ]
}
```

```

    },
    {
      "id": "AU_04",
      "type": "Layer",
      "name": "Business Layer",
      "description": "This layer is the core of the application and is responsible
for processing all the information.",
      "pageNumber": "41",
      "isPartOf": [],
    },
    {
      "id": "AU_05",
      "type": "Layer",
      "name": "Data Layer",
      "description": "It is the layer that is responsible for storing all the data
.",
      "pageNumber": "41",
      "isPartOf": [],
    },
    {
      "id": "AU_06",
      "type": "Service",
      "name": "API Gateway Service",
      "description": "This service is responsible for the management,
authentication and authorization of platform users.",
      "pageNumber": "44",
      "isPartOf": ["AU_04"],
    },
    {
      "id": "AU_07",
      "type": "Technology",
      "name": "Spring Boot",
      "description": "Backend framework used to implement the API Gateway Service
.",
      "pageNumber": "44",
      "isPartOf": ["AU_06"],
    },
    {
      "id": "AU_08",
      "type": "Connector",
      "description": "Presentation layer communicates with business layer",
      "pageNumber": "60",
      "isPartOf": ["AU_03", "AU_04"],
    }
  ],
  "patterns": [
    {
      "id": "P_01",
      "type": "Architectural Pattern",
      "name": "Client-Server",
      "description": "The platform is a web service that follows the client-server
architecture, where the client makes requests to a server that processes them
and returns the response.",
      "pageNumber": "41",
      "isPartOf": [],
      "fixedType": []
    }
  ]

```

```

    },
    {
      "id": "P_02",
      "type": "Architectural Pattern",
      "name": "Three Layers",
      "description": "The client-server model uses a 3-layer architecture, where
the system is divided into 3 layers.",
      "pageNumber": "41",
      "isPartOf": ["P_01"],
      "fixedType": []
    },
    {
      "id": "P_03",
      "type": "Architectural Pattern",
      "name": "Service-oriented",
      "description": "Both the business layer and the data layer are divided into
several different services.",
      "pageNumber": "42",
      "isPartOf": ["AU_04", "AU_05"],
      "fixedType": [Design pattern]
    },
    {
      "id": "P_04",
      "type": "Design Pattern",
      "name": "API Gateway",
      "description": "This design pattern allows only one component to interact
between users and the services provided by the platform.",
      "pageNumber": "43",
      "isPartOf": ["AU_02"],
      "fixedType": []
    }
  ]
}
...

```

Listing 4: Architectural Element Extraction Prompt v2

```

# Objective
You are an expert software architect and system design analyst. Extract the
Architectural Units (the concrete building blocks of the system) and patterns from the
provided software document.

# Instructions
1. Carefully read the entire document.
2. Identify architectural units and patterns using the type definitions below.
3. Extract the identified architectural units and patterns following the extraction
process defined below.

# Type Definitions
Below are the definitions for each type of architectural unit and pattern with their
concrete examples.

## Patterns

## Architectural Pattern
- Definition: A high-level structural organization of the system (e.g. Client-Server,
Layered Architecture, Microservices, MVVM, Service-oriented, Cloud Architecture).
- Examples: Layered Architecture, Hexagonal Architecture, API Gateway.

```

Design Pattern

- Definition: A lower-level software design solution used within units (e.g. API Gateway, Repository, Observer, Singleton, Shared Database, ORM, Component-based).
- Examples: Observer, Strategy, Factory.

Architectural Units

Layer

- Definition: a horizontal tier that groups units by a shared responsibility in a layered / n-tier architecture.
- Examples: presentation layer, business layer, data layer.

Component

- Definition: a concrete structural module of the system that either:
 1. represents an internal functional or structural part of the system that is not exposed as an independently running service; OR
 2. represents a technology-independent architectural role fulfilled by an infrastructure or data-management element.
- Examples: View, ViewModel, Model, Controller, Repository, Cache, Message Broker, Database.

Service

- Definition: a logical capability or independently running module of the system, including each microservice and each external / third-party service the system depends on, integrates with, or calls.
- Examples: API gateway, Microservice, Authentication Service, Payment Service.

Device

- Definition: a physical hardware endpoint or piece of equipment that participates in the system.
- Examples: a sensor, a screen / display, a kiosk or player device, a mobile or desktop device, an IoT device, a hardware appliance.

Technology

- Definition: a specific, named product, framework, library, programming language, protocol, cloud service, or development / testing / monitoring tool used to build or run the system, including a named third-party product that provides an external service or integration.
- Examples: PostgreSQL, React, AWS S3, Python, Spring Boot.

Connector

- Definition: a communication or interaction relationship among two or more Architectural Units. It can link any unit type to any unit type (e.g. layer-layer, service-service, service-database, device-service, component-service ...). A Connector has no name.
- One-to-one: a communication stated between two units.
- One-to-many: a single stated communication in which one unit communicates with several other units (e.g. a unit that routes, distributes, logs, or mediates communication among many units).

Other

- Definition: a participant in the system's architecture that does not fit any technical type above; might be:
 1. an external actor that interacts with the system; OR
 2. a high-level logical or structural part of the system that groups lower-level architectural elements but is not itself treated as a Layer, Component or Service.
- Examples: an end user, a client, "the backend", "the frontend".

Extraction Process

Patterns

- Assign each Pattern a strict sequential id: P_01, P_02, P_03...
- For each pattern, specify the page number from the document text (not the PDF page number) where it is described. If it spans several pages, list them all.
- For each pattern "description" must be the sentence or sentences from the document that states the pattern - the evidence proving the pattern.
- For each pattern, specify its type using exactly one of:
 - "isPartOf" is the list of OTHER pattern or unit ids this pattern belongs to. Leave it as an empty list when no parent pattern applies.
- If the source document misclassifies a pattern (e.g. a Design Pattern labelled as an Architectural Pattern), correct the classification and specify the changed type.
- Pattern should have the following JSON Schema:

```
{
  "id": "<Sequential Pattern id (P_01, P_02)>",
  "type": "<Architectural Pattern | Design Pattern>",
  "name": "<Name of the pattern in English>",
  "description": "<The exact document sentence(s) stating the pattern, translated to English>",
  "pageNumber": "<Page number(s) where the pattern is described>",
  "isPartOf": ["<id of the pattern this pattern is part of>"],
  "fixedType": "<If fixed, indicate the type before the fix>"
}
```

Architectural Units

- Assign each Architectural Unit a strict sequential id: AU_01, AU_02, AU_03...
- Specify its type using exactly one of: "Layer", "Component", "Service", "Device", "Technology", "Connector" or "Other".
- Specify description, sentence or sentences from the document that states the unit - the evidence proving the unit.
- Specify the page number from the document text (not the PDF page number) where it is described. If it spans several pages, list them all.
- For each unit except connector, "isPartOf" is the architectural unit or a pattern (architectural or design) this unit is contained by or belongs to. If there is no unit or pattern applies, leave it empty.
- For connector, "isPartOf" is the list of the TWO OR MORE Architectural Unit ids (AU_xx, taken from the provided units) that the Connector links. Include exactly the units the stated communication involves. For a one-to-many communication, list the central (hub) unit FIRST, followed by every unit it communicates with.
- Architectural Unit should have the following JSON Schema:

```
{
  "id": "<Sequential Architectural Unit id (AU_01, AU_02)>",
  "type": "<Layer | Component | Service | Device | Technology | Connector | Other>",
  "name": "<Name of the unit (If connector then empty) >",
  "description": "<The exact document sentence(s) stating the unit, translated to English>",
  "pageNumber": "<Page number(s) where the unit is described>",
  "isPartOf": ["<id of the unit this unit is part of>"],
}
```

Rules:

- Ensure every unit is strictly supported by the document; do not output a unit or technology whose name or role does not actually appear in the source, and include an

inferred unit only when the evidence is strong.

- Extract units from every view and section of the document (e.g. deployment, frontend structure, backend layering), not only from a single section.
- Extract each distinct unit individually, including units that are only listed together, named in passing, or mentioned in prose; never collapse several distinct units into one.
- Extract each real unit exactly once: do not output the same unit twice under different names, and do not split one real unit into several.
- When a structural or presentation pattern (e.g. MVC / MVVM) names its constituent parts, extract each named part as its own Component (View, ViewModel, Controller, View , etc.).
- Name each service by its capability or function (e.g. authentication, payment, storage), not by the product that implements it; when a named product provides the capability, additionally output that product as a separate Technology whose isPartOf is the Service.
 - Apply this especially to external / third-party integrations named by their product (e.g. a payment, email, storage, authentication, analytics, or monitoring provider). Output BOTH units, never the product alone:
 - a Service named by the capability it provides
 - a Technology named by the product, whose isPartOf is that Service.
- Extract every named technology, including ones mentioned only in the prose.
- Extract every named communication protocol (e.g. HTTP, HTTPS, REST, TCP, WebSocket, gRPC) as its own Technology, including protocols named only in passing or in the prose ; never omit a protocol as a mere implementation detail.
- Output should be given in JSON format as in the Example Output section.
- The whole output must be English. If the source document is in another language, translation should be made while extracting to ensure all extracted fields are in English.
 - During translating, avoid to paraphrase, summarize, reword, or normalize phrasing
- Avoid adding any extra explanation, just provide the required data.

Example Output (JSON)

```
{
  "architectural_units": [
    {
      "id": "AU_01",
      "type": "Other",
      "name": "Client",
      "description": "The client is the user of the platform.",
      "pageNumber": "41",
      "isPartOf": [],
    },
    {
      "id": "AU_02",
      "type": "Other",
      "name": "Backend",
      "description": "The Backend is the software that is responsible for processing user requests.",
      "pageNumber": "41",
      "isPartOf": [P_01],
    },
    {
      "id": "AU_03",
      "type": "Layer",
      "name": "Presentation Layer",
      "description": "It is the layer that is responsible for displaying information to the user.",
    }
  ]
}
```

```

    "pageNumber": "41",
    "isPartOf": [],
  },
  {
    "id": "AU_04",
    "type": "Layer",
    "name": "Business Layer",
    "description": "This layer is the core of the application and is responsible for processing all the information.",
    "pageNumber": "41",
    "isPartOf": [],
  },
  {
    "id": "AU_05",
    "type": "Layer",
    "name": "Data Layer",
    "description": "It is the layer that is responsible for storing all the data.",
    "pageNumber": "41",
    "isPartOf": [],
  },
  {
    "id": "AU_06",
    "type": "Service",
    "name": "API Gateway Service",
    "description": "This service is responsible for the management, authentication and authorization of platform users.",
    "pageNumber": "44",
    "isPartOf": ["AU_04"],
  },
  {
    "id": "AU_07",
    "type": "Technology",
    "name": "Spring Boot",
    "description": "Backend framework used to implement the API Gateway Service.",
    "pageNumber": "44",
    "isPartOf": ["AU_06"],
  },
  {
    "id": "AU_08",
    "type": "Connector",
    "description": "Presentation layer communicates with business layer",
    "pageNumber": "60",
    "isPartOf": ["AU_03", "AU_04"],
  }
],

"patterns": [
  {
    "id": "P_01",
    "type": "Architectural Pattern",
    "name": "Client-Server",
    "description": "The platform is a web service that follows the client-server architecture, where the client makes requests to a server that processes them and returns the response.",
    "pageNumber": "41",
    "isPartOf": [],
    "fixedType": []
  }
]

```

```

    },
    {
      "id": "P_02",
      "type": "Architectural Pattern",
      "name": "Three Layers",
      "description": "The client-server model uses a 3-layer architecture, where the
system is divided into 3 layers.",
      "pageNumber": "41",
      "isPartOf": ["P_01"],
      "fixedType": []
    },
    {
      "id": "P_03",
      "type": "Architectural Pattern",
      "name": "Service-oriented",
      "description": "Both the business layer and the data layer are divided into
several different services.",
      "pageNumber": "42",
      "isPartOf": ["AU_04", "AU_05"],
      "fixedType": []
    },
    {
      "id": "P_04",
      "type": "Design Pattern",
      "name": "API Gateway",
      "description": "This design pattern allows only one component to interact between
users and the services provided by the platform.",
      "pageNumber": "43",
      "isPartOf": ["AU_02"],
      "fixedType": []
    }
  ]
}
...

```

Listing 5: Architectural Element Extraction Prompt v3

```

# Objective
You are an expert software architect and system design analyst. Extract the
Architectural Units (the concrete building blocks of the system) and patterns from the
provided software document.

# Instructions
1. Carefully read the entire document.
2. Identify architectural units and patterns using the type definitions below.
3. Extract the identified architectural units and patterns following the extraction
process defined below.

# Type Definitions
Below are the definitions for each type of architectural unit and pattern with their
concrete examples.

## Patterns

## Architectural Pattern
- Definition: A high-level structural organization of the system (e.g. Client-Server,
Layered Architecture, Microservices, MVVM, Service-oriented, Cloud Architecture).

```

```

- Examples: Layered Architecture, Hexagonal Architecture, API Gateway.

## Design Pattern
- Definition: A lower-level software design solution used within units (e.g. API Gateway, Repository, Observer, Singleton, Shared Database, ORM, Component-based).
- Examples: Observer, Strategy, Factory.

## Architectural Units

### Layer
- Definition: a horizontal tier that groups units by a shared responsibility in a layered / n-tier architecture.
- Examples: presentation layer, business layer, data layer.

### Component
- Definition: a concrete structural module of the system that either:
  1. represents an internal functional or structural part of the system that is not exposed as an independently running service; OR
  2. represents a technology-independent architectural role fulfilled by an infrastructure or data-management element.
- Examples: View, ViewModel, Model, Controller, Repository, Cache, Message Broker, Database.

### Service
- Definition: a logical capability or independently running module of the system, including each microservice and each external / third-party service the system depends on, integrates with, or calls.
- Examples: API gateway, Microservice, Authentication Service, Payment Service.

### Device
- Definition: a physical hardware endpoint or piece of equipment that participates in the system.
- Examples: a sensor, a screen / display, a kiosk or player device, a mobile or desktop device, an IoT device, a hardware appliance.

### Technology
- Definition: a specific, named product, framework, library, programming language, protocol, cloud service, or development / testing / monitoring tool used to build or run the system, including a named third-party product that provides an external service or integration.
- Examples: PostgreSQL, React, AWS S3, Python, Spring Boot.

### Other
- Definition: a participant in the system's architecture that does not fit any technical type above; might be:
  1. an external actor that interacts with the system; OR
  2. a high-level logical or structural part of the system that groups lower-level architectural elements but is not itself treated as a Layer, Component or Service.
- Examples: an end user, a client, "the backend", "the frontend".

# Extraction Process

## Patterns

- Assign each Pattern a strict sequential id: P_01, P_02, P_03...
- For each pattern, specify the page number from the document text (not the PDF page number) where it is described. If it spans several pages, list them all.
- For each pattern "description" must be the sentence or sentences from the document

```

that states the pattern - the evidence proving the pattern.

- For each pattern, specify its type using exactly one of:
- "isPartOf" is the list of OTHER pattern or unit ids this pattern belongs to. Leave it as an empty list when no parent pattern applies.
- If the source document misclassifies a pattern (e.g. a Design Pattern labelled as an Architectural Pattern), correct the classification and specify the changed type.
- Pattern should have the following JSON Schema:

```
{
  "id": "<Sequential Pattern id (P_01, P_02)>",
  "type": "<Architectural Pattern | Design Pattern>",
  "name": "<Name of the pattern in English>",
  "description": "<The exact document sentence(s) stating the pattern, translated to English>",
  "pageNumber": "<Page number(s) where the pattern is described>",
  "isPartOf": ["<id of the pattern this pattern is part of>"],
  "fixedType": "<If fixed, indicate the type before the fix>"
}
```

Architectural Units

- Assign each Architectural Unit a strict sequential id: AU_01, AU_02, AU_03...
- Specify its type using exactly one of: "Layer", "Component", "Service", "Device", "Technology", or "Other".
- Specify description, sentence or sentences from the document that states the unit - the evidence proving the unit.
- Specify the page number from the document text (not the PDF page number) where it is described. If it spans several pages, list them all.
- For each unit, "isPartOf" is the architectural unit or a pattern (architectural or design) this unit is contained by or belongs to. If there is no unit or pattern applies, leave it empty.
- Architectural Unit should have the following JSON Schema:

```
{
  "id": "<Sequential Architectural Unit id (AU_01, AU_02)>",
  "type": "<Layer | Component | Service | Device | Technology | Other>",
  "name": "<Name of the unit>",
  "description": "<The exact document sentence(s) stating the unit, translated to English>",
  "pageNumber": "<Page number(s) where the unit is described>",
  "isPartOf": ["<id of the unit this unit is part of>"],
}
```

Rules:

- Ensure every unit is strictly supported by the document; do not output a unit or technology whose name or role does not actually appear in the source, and include an inferred unit only when the evidence is strong.
- Extract units from every view and section of the document (e.g. deployment, frontend structure, backend layering), not only from a single section.
- Extract each distinct unit individually, including units that are only listed together, named in passing, or mentioned in prose; never collapse several distinct units into one.
- Extract each real unit exactly once: do not output the same unit twice under different names, and do not split one real unit into several.
- When a structural or presentation pattern (e.g. MVC / MVVM) names its constituent parts, extract each named part as its own Component (View, ViewModel, Controller, View, etc.).
- Name each service by its capability or function (e.g. authentication, payment, storage), not by the product that implements it; when a named product provides the

capability, additionally output that product as a separate Technology whose isPartOf is the Service.

- Apply this especially to external / third-party integrations named by their product (e.g. a payment, email, storage, authentication, analytics, or monitoring provider). Output BOTH units, never the product alone:
 - a Service named by the capability it provides
 - a Technology named by the product, whose isPartOf is that Service.
- Extract every named technology, including ones mentioned only in the prose.
- Extract every named communication protocol (e.g. HTTP, HTTPS, REST, TCP, WebSocket, gRPC) as its own Technology, including protocols named only in passing or in the prose ; never omit a protocol as a mere implementation detail.
- Output should be given in JSON format as in the Example Output section.
- The whole output must be English. If the source document is in another language, translation should be made while extracting to ensure all extracted fields are in English.
- During translating, avoid to paraphrase, summarize, reword, or normalize phrasing
- Avoid adding any extra explanation, just provide the required data.

Example Output (JSON)

```
{
  "architectural_units": [
    {
      "id": "AU_01",
      "type": "Other",
      "name": "Client",
      "description": "The client is the user of the platform.",
      "pageNumber": "41",
      "isPartOf": [],
    },
    {
      "id": "AU_02",
      "type": "Other",
      "name": "Backend",
      "description": "The Backend is the software that is responsible for processing user requests.",
      "pageNumber": "41",
      "isPartOf": [P_01],
    },
    {
      "id": "AU_03",
      "type": "Layer",
      "name": "Presentation Layer",
      "description": "It is the layer that is responsible for displaying information to the user.",
      "pageNumber": "41",
      "isPartOf": [],
    },
    {
      "id": "AU_04",
      "type": "Layer",
      "name": "Business Layer",
      "description": "This layer is the core of the application and is responsible for processing all the information.",
      "pageNumber": "41",
      "isPartOf": [],
    }
  ]
}
```

```

    "id": "AU_05",
    "type": "Layer",
    "name": "Data Layer",
    "description": "It is the layer that is responsible for storing all the data.",
    "pageNumber": "41",
    "isPartOf": [],
  },
  {
    "id": "AU_06",
    "type": "Service",
    "name": "API Gateway Service",
    "description": "This service is responsible for the management, authentication
    and authorization of platform users.",
    "pageNumber": "44",
    "isPartOf": ["AU_04"],
  },
  {
    "id": "AU_07",
    "type": "Technology",
    "name": "Spring Boot",
    "description": "Backend framework used to implement the API Gateway Service.",
    "pageNumber": "44",
    "isPartOf": ["AU_06"],
  }
],
"patterns": [
  {
    "id": "P_01",
    "type": "Architectural Pattern",
    "name": "Client-Server",
    "description": "The platform is a web service that follows the client-server
    architecture, where the client makes requests to a server that processes them and
    returns the response.",
    "pageNumber": "41",
    "isPartOf": [],
    "fixedType": []
  },
  {
    "id": "P_02",
    "type": "Architectural Pattern",
    "name": "Three Layers",
    "description": "The client-server model uses a 3-layer architecture, where the
    system is divided into 3 layers.",
    "pageNumber": "41",
    "isPartOf": ["P_01"],
    "fixedType": []
  },
  {
    "id": "P_03",
    "type": "Architectural Pattern",
    "name": "Service-oriented",
    "description": "Both the business layer and the data layer are divided into
    several different services.",
    "pageNumber": "42",
    "isPartOf": ["AU_04", "AU_05"],
    "fixedType": [Design Pattern]
  }
],

```

```

    {
      "id": "P_04",
      "type": "Design Pattern",
      "name": "API Gateway",
      "description": "This design pattern allows only one component to interact between
        users and the services provided by the platform.",
      "pageNumber": "43",
      "isPartOf": ["AU_02"],
      "fixedType": []
    }
  ]
}

...

```

Listing 6: Architectural Element Extraction Prompt v4

```

# Objective
You are an expert software architect and system design analyst. Extract the
Architectural Units (the concrete building blocks of the system) and patterns from the
provided software document.

# Instructions
1. Carefully read the entire document.
2. Identify architectural units and patterns using the type definitions below.
3. Extract the identified architectural units and patterns following the extraction
   process defined below.

# Type Definitions
Below are the definitions for each type of architectural unit and pattern with their
concrete examples.

## Patterns

## Architectural Pattern
- Definition: A high-level structural organization of the system (e.g. Client-Server,
  Layered Architecture, Microservices, MVVM, Service-oriented, Cloud Architecture).
- Examples: Layered Architecture, Hexagonal Architecture, API Gateway.

## Design Pattern
- Definition: A lower-level software design solution used within units (e.g. API
  Gateway, Repository, Observer, Singleton, Shared Database, ORM, Component-based).
- Examples: Observer, Strategy, Factory.

## Architectural Units

### Layer
- Definition: a horizontal tier that groups units by a shared responsibility in a
  layered / n-tier architecture.
- Examples: presentation layer, business layer, data layer.

### Component
- Definition: a concrete structural module of the system that either:
  1. represents an internal functional or structural part of the system that is not
    exposed as an independently running service; OR
  2. represents a technology-independent architectural role fulfilled by an
    infrastructure or data-management element.
- Examples: View, ViewModel, Model, Controller, Repository, Cache, Message Broker,
  Database.

```

Service

- Definition: a logical capability or independently running module of the system, including each microservice and each external / third-party service the system depends on, integrates with, or calls.
- Examples: API gateway, Microservice, Authentication Service, Payment Service.

Device

- Definition: a physical hardware endpoint or piece of equipment that participates in the system.
- Examples: a sensor, a screen / display, a kiosk or player device, a mobile or desktop device, an IoT device, a hardware appliance.

Technology

- Definition: a specific, named product, framework, library, programming language, protocol, cloud service, or development / testing / monitoring tool used to build or run the system, including a named third-party product that provides an external service or integration.
- Examples: PostgreSQL, React, AWS S3, Python, Spring Boot.

Other

- Definition: a participant in the system's architecture that does not fit any technical type above; might be:
 1. an external actor that interacts with the system; OR
 2. a high-level logical or structural part of the system that groups lower-level architectural elements but is not itself treated as a Layer, Component or Service.
- Examples: an end user, a client, "the backend", "the frontend".

Extraction Process

Patterns

- Assign each Pattern a strict sequential id: P_01, P_02, P_03...
- For each pattern, specify the page number from the document text (not the PDF page number) where it is described. If it spans several pages, list them all.
- For each pattern "description" must be the sentence or sentences from the document that states the pattern - the evidence proving the pattern.
- For each pattern, specify its type using exactly one of:
 - "isPartOf" is the list of pattern or unit ids this pattern belongs to. Leave it as an empty list when no parent pattern or unit applies.
 - If the source document misclassifies a pattern (e.g. a Design Pattern labelled as an Architectural Pattern), correct the classification and specify the changed type.
- Pattern should have the following JSON Schema:

```
{
  "id": "<Sequential Pattern id (P_01, P_02)>",
  "type": "<Architectural Pattern | Design Pattern>",
  "name": "<Name of the pattern in English>",
  "description": "<The exact document sentence(s) stating the pattern, translated to English>",
  "pageNumber": "<Page number(s) where the pattern is described>",
  "isPartOf": ["<id of the pattern or unit this pattern is part of>"],
  "fixedType": "<If fixed, indicate the type before the fix>"
}
```

Architectural Units

- Assign each Architectural Unit a strict sequential id: AU_01, AU_02, AU_03...

- Specify its type using exactly one of: "Layer", "Component", "Service", "Device", "Technology", or "Other".
- Specify description, sentence or sentences from the document that states the unit - the evidence proving the unit.
- Specify the page number from the document text (not the PDF page number) where it is described. If it spans several pages, list them all.
- For each unit, "isPartOf" is the single most specific container that applies, in this order:
 - (1) the pattern it participates in
 - (2) its Layer
 - (3) the Service or Component that uses it
 - (4) the high-level unit it belongs to (e.g. Frontend, Backend, Server, Mobile Application)
 - (5) empty if none applies.
 - For a Technology, start at (3).
 - List several parents only when the document states them.
- Architectural Unit should have the following JSON Schema:


```
{
  "id": "<Sequential Architectural Unit id (AU_01, AU_02)>",
  "type": "<Layer | Component | Service | Device | Technology | Other>",
  "name": "<Name of the unit>",
  "description": "<The exact document sentence(s) stating the unit, translated to English>",
  "pageNumber": "<Page number(s) where the unit is described>",
  "isPartOf": ["<id of the pattern or unit this unit is part of>"],
}
```

Rules:

- Ensure every unit is strictly supported by the document; do not output a unit or technology whose name or role does not actually appear in the source, and include an inferred unit only when the evidence is strong
- Extract units from every view and section of the document (e.g. deployment, frontend structure, backend layering), not only from a single section.
- Extract each distinct unit individually, including units that are only listed together, named in passing, or mentioned in prose; never collapse several distinct units into one.
- Extract each real unit exactly once: do not output the same unit twice under different names, and do not split one real unit into several.
- When a structural or presentation pattern (e.g. MVC / MVVM) names its constituent parts, extract each named part as its own Component (View, ViewModel, Controller, View, etc.).
- Name each service by its capability or function (e.g. authentication, payment, storage), not by the product that implements it; when a named product provides the capability, additionally output that product as a separate Technology whose isPartOf is the Service.
 - Apply this especially to external / third-party integrations named by their product (e.g. a payment, email, storage, authentication, analytics, or monitoring provider). Output BOTH units, never the product alone:
 - a Service named by the capability it provides
 - a Technology named by the product, whose isPartOf is that Service.
- Extract every named technology, including ones mentioned only in the prose.
- Extract every named communication protocol (e.g. HTTP, HTTPS, REST, TCP, WebSocket, gRPC) as its own Technology, including protocols named only in passing or in the prose; never omit a protocol as a mere implementation detail.
- Output should be given in JSON format as in the Example Output section.
- The whole output must be English. If the source document is in another language, translation should be made while extracting to ensure all extracted fields are in English.

```

- During translating, avoid to paraphrase, summarize, reword, or normalize phrasing
.
- Avoid adding any extra explanation, just provide the required data.

# Example Output (JSON)
{
  "architectural_units": [
    {
      "id": "AU_01",
      "type": "Other",
      "name": "Client",
      "description": "The client is the user of the platform.",
      "pageNumber": "41",
      "isPartOf": [P_01],
    },
    {
      "id": "AU_02",
      "type": "Other",
      "name": "Backend",
      "description": "The Backend is the software that is responsible for processing
user requests.",
      "pageNumber": "41",
      "isPartOf": [P_01],
    },
    {
      "id": "AU_03",
      "type": "Layer",
      "name": "Presentation Layer",
      "description": "It is the layer that is responsible for displaying information to
the user.",
      "pageNumber": "41",
      "isPartOf": [P_02],
    },
    {
      "id": "AU_04",
      "type": "Layer",
      "name": "Business Layer",
      "description": "This layer is the core of the application and is responsible for
processing all the information.",
      "pageNumber": "41",
      "isPartOf": [P_02],
    },
    {
      "id": "AU_05",
      "type": "Layer",
      "name": "Data Layer",
      "description": "It is the layer that is responsible for storing all the data.",
      "pageNumber": "41",
      "isPartOf": [P_02],
    },
    {
      "id": "AU_06",
      "type": "Service",
      "name": "API Gateway Service",
      "description": "This service is responsible for the management, authentication
and authorization of platform users.",
      "pageNumber": "44",
      "isPartOf": ["AU_04"],
    }
  ]
}

```

```

    },
    {
      "id": "AU_07",
      "type": "Technology",
      "name": "Spring Boot",
      "description": "Backend framework used to implement the API Gateway Service.",
      "pageNumber": "44",
      "isPartOf": ["AU_06"],
    }
  ],
  "patterns": [
    {
      "id": "P_01",
      "type": "Architectural Pattern",
      "name": "Client-Server",
      "description": "The platform is a web service that follows the client-server architecture, where the client makes requests to a server that processes them and returns the response.",
      "pageNumber": "41",
      "isPartOf": [],
      "fixedType": []
    },
    {
      "id": "P_02",
      "type": "Architectural Pattern",
      "name": "Three Layers",
      "description": "The client-server model uses a 3-layer architecture, where the system is divided into 3 layers.",
      "pageNumber": "41",
      "isPartOf": [],
      "fixedType": []
    },
    {
      "id": "P_03",
      "type": "Architectural Pattern",
      "name": "Service-oriented",
      "description": "Both the business layer and the data layer are divided into several different services.",
      "pageNumber": "42",
      "isPartOf": ["AU_04", "AU_05"],
      "fixedType": [Design Pattern]
    },
    {
      "id": "P_04",
      "type": "Design Pattern",
      "name": "API Gateway",
      "description": "This design pattern allows only one component to interact between users and the services provided by the platform.",
      "pageNumber": "43",
      "isPartOf": ["AU_02"],
      "fixedType": []
    }
  ]
}
...

```

Listing 7: Architectural Element Extraction Prompt v3

```

# Objective
You are an expert software architect and system design analyst. Given the already-
extracted Architectural Units (provided below as JSON) and the software document,
extract the Connectors of the system - the communications between its Architectural
Units.

# Instructions
1. Carefully read the entire document.
2. Extract connectors and technologies used from document using the type definition
and extracting process below.
3. Check if technology or protocol specified for the connector, if yes, extract it
separately as a technology following the definition and extraction process.

## What is connector
- Definition: a communication or interaction relationship among two or more
Architectural Units. It can link any unit type to any unit type (e.g. layer-layer,
service-service, service-database, device-service, component-service ...).
- One-to-one: a communication stated between two units.
- One-to-many: a single stated communication in which one unit communicates with
several other units (e.g. a unit that routes, distributes, logs, or mediates
communication among many units).

## What is technology
- Definition: Protocols or technologies used by the connector (in the communication
)
- Examples: HTTP, REST, gRPC etc.

# Extraction Process

## Connector
- Assign each connector a strict sequential id: C_01, C_02, C_03....
- "isPartOf" is the list of the TWO OR MORE Architectural Unit ids (AU_xx, taken from
the provided units) that the Connector links.
  - Include exactly the units the stated communication involves.
  - For a one-to-many communication, list the central (hub) unit FIRST, followed by
every unit it communicates with.
- "description" must be the sentence or sentences from the document that states the
communication - the evidence proving the connection.
  - Some keywords to look for, communicate, connect, send, receive etc.
  - In case of multiple sentences from different pages, sentences should be separated
with "[...]"
- Specify the page number from the document text (not the PDF page number) where the
communication is described. If it spans several pages, list them all.
- Connector should have the following JSON Schema:
  {
    "id": "<Sequential connector id (C_01, C_02)>",
    "type": "Connector",
    "description": "<The exact document sentence(s) stating the communication,
translated to English>",
    "pageNumber": "<Page number(s) where the communication is described>",
    "isPartOf": ["<id of a linked unit>", "<id of another linked unit>", "..."],
  }

## Technology
- Assign each Technology a strict sequential id: T_01, T_02, T_03....
- "name" is the name of the protocol/technology exactly as stated (or its standard

```

```

name if the document uses an abbreviation/variant), e.g. "HTTP", "REST", "gRPC".
- "isPartOf" is the list of Connector ids (C_xx, from the extracted Connectors) that
use this technology.
- "description" must be the exact sentence(s) from the document stating that this
technology/protocol is used for the communication - the evidence proving the
technology usage.
- Specify the page number(s) from the document text (not the PDF page number) where
the technology is mentioned.
- Technology should have the following JSON Schema:
{
  "id": "<Sequential technology id (T_01, T_02)>",
  "type": "Technology",
  "name": "<Name of the technology as stated in the document>",
  "description": "<The exact document sentence(s) stating the technology is used,
translated to English>",
  "pageNumber": "<Page number(s) where the technology is described>",
  "isPartOf": ["<id of a connector using this technology>"],
}

# Rules:
- A Connector links two or more units; put every unit the stated communication
involves in "isPartOf".
- When a single statement describes one unit communicating with a set of units in the
same way (one-to-many - e.g. a unit that routes, distributes, logs, or mediates among
many units), output ONE Connector with that central unit listed FIRST, followed by
every unit in the set; do not split it into separate pairs.
- Output a separate Connector for each independently stated communication; do not
merge communications the document states separately into one Connector.
- Do not extract the individual steps of a use-case, scenario, or action sequence as
connectors; capture only the structural communications between the architectural units
involved.
- A Connector may link any unit type to any unit type.
- When one statement describes a communication at several levels (e.g. a layer
communicating with another layer, and more specifically with a service inside it),
output a separate Connector for EACH stated level; do not collapse them into one.
- Reference only ids from the provided Architectural Units; if an endpoint you see is
not in the list, avoid extracting.
- Only extract technologies related to the connector and it is explicitly stated by
the document.
- Ensure every connector is strictly supported by the document; do not invent
communications. Do not infer a connector merely because two units could plausibly
interact (for example, do not connect every service to a shared database or
infrastructure unit) - extract only the communications the document explicitly states.
- Output should be given in JSON format as in the Example Output section.
- The whole output must be English. If the source document is in another language,
translation should be made while extracting to ensure all extracted fields are in
English.
  - During translating, avoid to paraphrase, summarize, reword, or normalize phrasing
  .

# Example Output (JSON)
{
  "connectors": [
    {
      "id": "C_01",
      "type": "Connector",
      "description": "The presentation layer communicates with the business layer.
[...] presentation layer communicates with business layer for calling the

```

```

    necessary functions.",
    "pageNumber": "42,61",
    "isPartOf": ["AU_03", "AU_04"],
  },
  {
    "id": "C_02",
    "type": "Connector",
    "description": "The gateway service routes every incoming request to the
    microservice A, B, C using gRPC.",
    "pageNumber": "44",
    "isPartOf": ["AU_06", "AU_09", "AU_10", "AU_11"],
  },
  {
    "id": "T_01",
    "type": "Technology",
    "name": "gRPC",
    "description": "The gateway service routes every incoming request to the
    microservice A, B, C using gRPC.",
    "pageNumber": "44",
    "isPartOf": ["C_02"],
  }
]
}
...

```

.3 Stage III: Architectural Decision Extraction Prompt

Listing 8: Architectural Decision Extraction Prompt v1

```

# Objective
You are an expert software architect and requirements engineer. Given the already-
extracted Requirements (and Concepts) and the already-extracted Architecture (
Architectural Units and Patterns), both provided below as JSON, and the software
document, extract the Architectural Decisions of the system.

# Instructions
1. Carefully read the entire document.
2. Identify the architectural decisions in the document according to the definition
and rules given below.
3. Extract each of them according to the Extraction Process and rules defined below.

## Architectural Decision
- Definition: a stated link between ONE OR MORE architectural elements (units and
patterns) and ONE OR MORE requirement or concept that motivated it, evidenced by a
rationale sentence explaining why the element was chosen, or what it achieves, with
respect to those requirements/concepts.

# Extraction Process
- Assign each Architectural Decision a strict sequential id: AD_01, AD_02, AD_03...
- "architecturalElementIds" should indicate which architectural elements (units or
patterns) decision is related to.
- "architecturalDecisionSource" must be a list contains:
  - Related Requirement/Concept Id's (R_xx or C_xx).
  - If the rationale introduces a motivating concept that is not present in the
provided concepts, include the concept directly as an English string rather than

```

```

    assigning it an ID.
- "rationale" must be the text from the document translated to English that state why
the architectural element addresses the requirement/concept - the evidence proving the
decision.
- Specify the page number from the document text (not the PDF page number) where the
rationale is stated. If it spans several pages, list them all.
- Architectural Decision should have the following JSON Schema:
{
  "id": "<Sequential Architectural Decision id (AD_01, AD_02)>",
  "architecturalElementIds": [<ids of the related units and patterns (AU_xx or P_xx)
>],
  "architecturalDecisionSource": [<ids of the related requirement or extracted
concept from rationale>],
  "rationale": "<The document sentence(s) justifying the decision, translated to
English>",
  "pageNumber": "<Page number(s) where the rationale is stated>"
}

# Rules:
- Output should be given in JSON format as in the Example Output section.
- The whole output must be English. Translation must be made while extracting to
ensure all extracted fields are in English.
  - During translating, avoid to paraphrase, summarize, reword, or normalize phrasing
  .

# Example Output (JSON)
{
  "architectural_decisions": [
    {
      "id": "AD_01",
      "architecturalElementIds": ["AU_10", "AU_11"],
      "architecturalDecisionSource": ["C_01", "Scalability"],
      "rationale": "Increases system security [...] It is only the Api Gateway that is
responsible for authorization and SSL with the client",
      "pageNumber": "43"
    },
    {
      "id": "AD_02",
      "architecturalElementIds": ["P_01"],
      "architecturalDecisionSource": ["C_05"],
      "rationale": "The decision was made to use AWS and not other cloud services due
to its low price and its previous use by the company.",
      "pageNumber": "46"
    }
  ]
}
...

```

Listing 9: Architectural Decision Extraction Prompt v2

```

# Objective
You are an expert software architect and requirements engineer. Given the already-
extracted Requirements (and Concepts) and the already-extracted Architecture (
Architectural Units and Patterns), both provided below as JSON, and the software
document, extract the Architectural Decisions of the system.

# Instructions

```

```

1. Carefully read the entire document.
2. Identify the architectural decisions in the document according to the definition
and rules given below.
3. Extract each of them according to the Extraction Process and rules defined below.

## Architectural Decision
- Definition: a stated link between the architectural element(s) that one
justification in the document is about, and the requirement(s) or concept(s) that
motivated it.
- Identification question: Does this sentence say why this system uses it?
- Meta Examples:
  - Enablement - "The <element> allows <capability>, facilitating the <quality> of
the application."
  - Explicit choice with reason - "<element> was chosen over <alternative> due to its
<property>."
  - Purpose-first - "In order to <achieve quality>, <element> has been used."
  - Role - "<element> is responsible for <function>, ensuring <quality>."

# Extraction Process
- Assign each Architectural Decision a strict sequential id: AD_01, AD_02, AD_03...
- "architecturalElementIds" should indicate which architectural elements (units or
patterns) decision is related to.
- "architecturalDecisionSource" must be a list contains:
  - Related Requirement/Concept Id's (R_xx or C_xx).
  - If the rationale introduces a motivating concept that is not present in the
provided concepts, include the concept directly as an English string rather than
assigning it an ID.
- "rationale" must be the text from the document translated to English that state why
the architectural element addresses the requirement/concept - the evidence proving the
decision.
- Specify the page number from the document text (not the PDF page number) where the
rationale is stated. If it spans several pages, list them all.
- Architectural Decision should have the following JSON Schema:
  {
    "id": "<Sequential Architectural Decision id (AD_01, AD_02)>",
    "architecturalElementIds": [<ids of the related units and patterns (AU_xx or P_xx)
>],
    "architecturalDecisionSource": [<ids of the related requirement or extracted
concept from rationale>],
    "rationale": "<The document sentence(s) justifying the decision, translated to
English>",
    "pageNumber": "<Page number(s) where the rationale is stated>"
  }

# Rules:
- If the document justifies an element in several separate places, that is several
decisions, one per justification.
- Output should be given in JSON format as in the Example Output section.
- The whole output must be English. Translation must be made while extracting to
ensure all extracted fields are in English.
  - During translating, avoid to paraphrase, summarize, reword, or normalize phrasing
  .

# Example Output (JSON)
{
  "architectural_decisions": [
    {
      "id": "AD_01",

```

```

    "architecturalElementIds": ["AU_10", "AU_11"],
    "architecturalDecisionSource": ["C_01", "Scalability"],
    "rationale": "Using this cloud provider increases scalability and maintainability
    .",
    "pageNumber": "38"
  },
  {
    "id": "AD_02",
    "architecturalElementIds": ["P_01"],
    "architecturalDecisionSource": ["C_05"],
    "rationale": "This pattern enhances security and confidentiality",
    "pageNumber": "46"
  }
]
}
...

```

Listing 10: Architectural Decision Extraction Prompt v3

```

# Objective
You are an expert software architect and requirements engineer. Given the already-
extracted Requirements (and Concepts) and the already-extracted Architecture (
Architectural Units and Patterns), both provided below as JSON, and the software
document, extract the Architectural Decisions of the system.

# Instructions
1. Carefully read the entire document.
2. Go through the provided architecture JSON and for each of them locate every passage
   in the document that mentions it.
3. In those passages, collect go over each sentence and identify the ones that states
   a benefit, purpose, quality, goal or reason for that element (Check the type
   definition below for details).
4. Extract each of them according to the Extraction Process and rules defined below.

## Architectural Decision
- Definition: A sentence demonstrates a stated link in the document between the
architectural element(s) and the requirement(s) or concept(s) that motivated it.
- Identification Questions (One yes out of three is enough to accept):
  - Does this sentence say why this system uses it?
  - Does this sentence state something an architectural element does or provides,
together with what that improves, simplifies, ensures, protects, facilitates or
enables?
  - Does the sentence describe the element with a word that is itself a quality (
compatible, portable, scalable, efficient, flexible, open source)?
- Meta Examples:
  - Enablement - "The <element> allows <capability>, facilitating the <quality> of
the application."
  - Explicit choice with reason - "<element> was chosen over <alternative> due to its
<property>."
  - Purpose-first - "In order to <achieve quality>, <element> has been used."
  - Role - "<element> is responsible for <function>, ensuring <quality>."

# Extraction Process
- Assign each Architectural Decision a strict sequential id: AD_01, AD_02, AD_03...
- "architecturalElementIds" should indicate which architectural elements (units or
patterns) decision is related to.
- "architecturalDecisionSource" must be a list contains:

```

```

- Related Requirement/Concept Id's (R_xx or C_xx).
- If the rationale introduces a motivating concept that is not present in the
  provided concepts, include the concept directly as an English string rather than
  assigning it an ID.
- "rationale" must be the text from the document translated to English that state why
  the architectural element addresses the requirement/concept - the evidence proving the
  decision.
- Specify the page number from the document text (not the PDF page number) where the
  rationale is stated. If it spans several pages, list them all.
- Architectural Decision should have the following JSON Schema:
  {
    "id": "<Sequential Architectural Decision id (AD_01, AD_02)>",
    "architecturalElementIds": [<ids of the related units and patterns (AU_xx or P_xx)
    >],
    "architecturalDecisionSource": [<ids of the related requirement or extracted
    concept from rationale>],
    "rationale": "<The document sentence(s) justifying the decision, translated to
    English>",
    "pageNumber": "<Page number(s) where the rationale is stated>"
  }

# Rules:
- Each sentence mentioning or referring a concept (a quality) and architectural
  element is a strong candidate for architectural decision.
- If the document justifies an element in several separate places, that is several
  decisions, one per justification.
- The unit of extraction is the justifying sentence, NOT the architectural element.
  One element may have several decisions.
- Output should be given in JSON format as in the Example Output section.
- The whole output must be English. Translation must be made while extracting to
  ensure all extracted fields are in English.
  - During translating, avoid to paraphrase, summarize, reword, or normalize phrasing
  .

# Example Output (JSON)
{
  "architectural_decisions": [
    {
      "id": "AD_01",
      "architecturalElementIds": ["AU_10", "AU_11"],
      "architecturalDecisionSource": ["C_01", "Scalability"],
      "rationale": "Using this cloud provider increases scalability and maintainability
      .",
      "pageNumber": "38"
    },
    {
      "id": "AD_02",
      "architecturalElementIds": ["P_01"],
      "architecturalDecisionSource": ["C_05"],
      "rationale": "This pattern enhances security and confidentiality",
      "pageNumber": "46"
    }
  ]
}
...

```

Listing 11: Architectural Decision Extraction Prompt v4

Objective

You are an expert software architect and requirements engineer. Given the already-extracted Architecture (Architectural Units and Patterns), and requirements/concepts both provided below as JSON, and the software document, extract the Architectural Decisions of the system.

Instructions

1. Carefully read the entire document.
2. Go through the provided architecture JSON element by element. For each architectural element (unit or pattern), locate every passage in the document that mentions it.
3. In those passages, collect EVERY separate sentence that states a benefit, purpose, quality, goal or reason for that element (Check the type definition below for details).
4. Extract each of them according to the Extraction Process and rules defined below.

Architectural Decision

- Definition: A sentence demonstrates a stated link in the document between the architectural element(s) and the requirements or concepts (qualities of the element) that motivated it.
- Identification Questions (One yes out of three is enough to accept):
 - Does this sentence say why this system uses it?
 - Does this sentence state something an architectural element does or provides, together with what that improves, simplifies, ensures, protects, facilitates or enables?
 - Does the sentence describe the element with a word that is itself a quality (compatible, portable, scalable, efficient, flexible, open source)?
- Meta Examples:
 - Enablement - "The <element> allows <capability>, facilitating the <quality> of the application."
 - Explicit choice with reason - "<element> was chosen over <alternative> due to its <property>."
 - Purpose-first - "In order to <achieve quality>, <element> has been used."
 - Role - "<element> is responsible for <function>, ensuring <quality>."

Extraction Process

- Assign each Architectural Decision a strict sequential id: AD_01, AD_02, AD_03...
- "architecturalElementIds" should indicate which architectural elements (units or patterns) decision is related to.
- "rationale" must be the text from the document translated to English that state why the architectural element addresses the requirement/concept - the evidence proving the decision.
- Specify the page number from the document text (not the PDF page number) where the rationale is stated. If it spans several pages, list them all.
- Architectural Decision should have the following JSON Schema:


```
{
  "id": "<Sequential Architectural Decision id (AD_01, AD_02)>",
  "architecturalElementIds": [<ids of the related units and patterns (AU_xx or P_xx)>],
  "rationale": "<The document sentence(s) justifying the decision, translated to English>",
  "pageNumber": "<Page number(s) where the rationale is stated>"
}
```

Rules:

- Architectural Decisions should not have to be related with the given concepts/requirements necessarily, they can be related to other feature or quality not covered

```

by the JSON, so use them only for reference.
- Each sentence mentioning or referring a concept (a quality) and architectural
element is a strong candidate for architectural decision.
- If the document justifies an element in several separate places, that is several
decisions, one per justification.
- The unit of extraction is the justifying sentence, NOT the architectural element.
One element may have several decisions.
- Output should be given in JSON format as in the Example Output section.
- The whole output must be English. Translation must be made while extracting to
ensure all extracted fields are in English.
    - During translating, avoid to paraphrase, summarize, reword, or normalize phrasing
    .

# Example Output (JSON)
{
  "architectural_decisions": [
    {
      "id": "AD_01",
      "architecturalElementIds": ["AU_10", "AU_11"],
      "rationale": "Using this cloud provider increases scalability and maintainability
.",
      "pageNumber": "38"
    },
    {
      "id": "AD_02",
      "architecturalElementIds": ["P_01"],
      "rationale": "This pattern enhances security and confidentiality",
      "pageNumber": "46"
    }
  ]
}
...

```

Listing 12: Architectural Decision Source Extraction Prompt v4

```

# Objective
You are an expert software architect and requirements engineer. Given the already-
extracted Requirements and Concepts, the already-extracted Architecture (Architectural
Units and Patterns) and the already-extracted Architectural Decisions, all provided
below as JSON, extract the source of every Architectural Decision.

# Instructions
1. Go through the provided architectural decisions one by one.
2. For each decision, read its rationale and identify every requirement or concept (
quality) the rationale states as the motivation for the architectural element(s) it
references.
3. Match each of them against the provided requirements and concepts, and create a
concept for the ones none of the provided concepts covers.
4. Extract each of them according to the Extraction Process and rules defined below.

## Architectural Decision Source
- Definition: the requirement(s) or concept(s) that the rationale of an architectural
decision names as the motivation for the architectural element(s) of that decision.
- Identification Question: Which of the provided requirements/concepts does this
rationale say the element achieves, improves, ensures, protects, facilitates or
enables?

```

```
# Extraction Process
- Keep the "id" of the decision exactly as it is provided. Never renumber, merge,
split, add or remove a decision.
- "architecturalDecisionSource" must be a list contains:
  - Related Requirement/Concept Id's (R_xx or C_xx). A match is by meaning, the
  wording of the rationale and of the requirement/concept does not have to be
  identical.
  - If the rationale introduces a motivating concept that is not present in the
  provided concepts, include the concept directly as an English string rather than
  assigning it an ID.
- Architectural Decision Source should have the following JSON Schema:
  {
    "id": "<The provided Architectural Decision id (AD_01, AD_02)>",
    "architecturalDecisionSource": [<ids of the related requirement or extracted
    concept from rationale>]
  }

# Rules:
- Every provided decision must appear in the output exactly once, in the order it is
provided.
- Only what the rationale itself states counts as a source. Never infer a motivation
from the name or the description of the architectural element.
- When the rationale introduces a motivating concept not present in the provided
concepts, add it as a short English quality-attribute noun - "Scalability", "Security
", "Maintainability", "Performance" - matching the style of the provided concepts.
Never use a technology, product or component name as a concept.
- One rationale may state several motivations, so a decision may have several sources.
List each of them once.
- Output should be given in JSON format as in the Example Output section.
- The whole output must be English.

# Example Output (JSON)
{
  "architectural_decisions": [
    {
      "id": "AD_01",
      "architecturalDecisionSource": ["C_01", "Scalability"]
    },
    {
      "id": "AD_02",
      "architecturalDecisionSource": ["C_05"]
    },
    {
      "id": "AD_03",
      "architecturalDecisionSource": ["C_01", "C_02", "Maintainability"]
    }
  ]
}
...
```